Design and Optimization of a Custom Matrix Multiplication Accelerator using Vitis HLS on Zynq UltraScale+

Braylon Watson
DSA

April 26, 2026

Abstract

This project investigates hardware acceleration of matrix multiplication using AMD Vitis High-Level Synthesis (HLS) targeting the Zynq UltraScale+ MPSoC platform. A baseline accelerator from Lab 3 was extended into a custom accelerator using loop pipelining, loop unrolling, tiled local buffering, and array partitioning. The resulting design was synthesized and compared against previous CPU-based Lab 2 software implementations and the original HLS baseline design. Results demonstrate the classic FPGA tradeoff of increased hardware resource utilization in exchange for higher parallelism and performance potential.

1 Introduction

Matrix multiplication is a fundamental kernel used in scientific computing, machine learning, graphics, and signal processing. Because of its high computational cost, it is often used to evaluate processor and accelerator performance.

This project builds on three previous labs:

- **Lab 1:** Embedded software execution on the ARM processor of the Zynq platform.
- **Lab 2:** CPU-based matrix multiplication benchmarking using multiple software optimization methods.
- **Lab 3:** Hardware acceleration using AMD Vitis HLS.

For the final project, a custom accelerator was developed based on Lab 3 and compared against Lab 2 CPU implementations.

2 Background

2.1 Lab 2 CPU Implementations

Three software matrix multiplication versions were tested on the ARM CPU:

1. Naive triple-loop multiplication
2. Cache-aware loop reordering
3. Tiled/block-based multiplication

These optimizations improved memory locality and cache performance.

2.2 Lab 3 Baseline HLS Accelerator

The baseline Lab 3 design used Vitis HLS to convert C/C++ matrix multiplication code into synthesizable hardware logic with AXI interfaces.

3 Custom Accelerator Design

The final project extends the baseline HLS design with additional optimizations:

- **Loop Pipelining** to overlap operations
- **Loop Unrolling** for parallel multiply-accumulate units
- **Tiled Computation** for local data reuse
- **Array Partitioning** for parallel memory access
- **Multiple AXI Memory Ports** for concurrent reads/writes

These techniques increase throughput at the cost of more FPGA resources.

4 Implementation

The design was implemented in AMD Vitis HLS targeting:

- Device: `xczu3eg-sbva484-1-e`
- Family: Zynq UltraScale+ MPSoC
- Tool Version: AMD Vitis 2025.2

5 Results

5.1 Resource Utilization

Table 1: Baseline vs Custom Accelerator Resource Comparison

Resource	Baseline Lab 3	Custom Accelerator
BRAM	2	4
DSP	7	60
FF	2,190	16,767
LUT	2,331	11,617
URAM	0	0

5.2 CPU Benchmark Results from Lab 2

Table 2: Lab 2 CPU Matrix Multiplication Results

Method	N=16 GFLOPS	N=32 GFLOPS
Naive	0.0313	0.0319
Cache-aware	0.0388	0.0398
Tiled	0.0363	0.0364

6 Discussion

The custom accelerator consumed significantly more hardware resources than the baseline Lab 3 implementation. This increase is expected because loop unrolling and parallel execution require additional DSP slices, LUTs, and registers.

Compared to CPU implementations from Lab 2, the FPGA accelerator offers a fundamentally different execution model:

- CPU executes instructions sequentially or with limited parallelism.
- FPGA accelerator performs many operations simultaneously in hardware.

This demonstrates the primary advantage of FPGA-based acceleration for compute-intensive kernels.

7 Conclusion

A custom matrix multiplication accelerator was successfully designed using Vitis HLS and synthesized for the Zynq UltraScale+ platform. By applying tiling, pipelining, loop unrolling, and memory optimizations, the design increased available parallelism compared to the baseline implementation.

Although hardware resource utilization increased, the project demonstrates how FPGAs can trade silicon area for performance. This is especially useful for workloads such as linear algebra and machine learning inference.

8 Future Work

Possible future improvements include:

- Larger systolic-array architecture
- Support for floating-point arithmetic
- DMA-based host transfers
- Real hardware benchmarking on-board
- Comparison against GPU execution

Appendix

Synthesis Screenshot

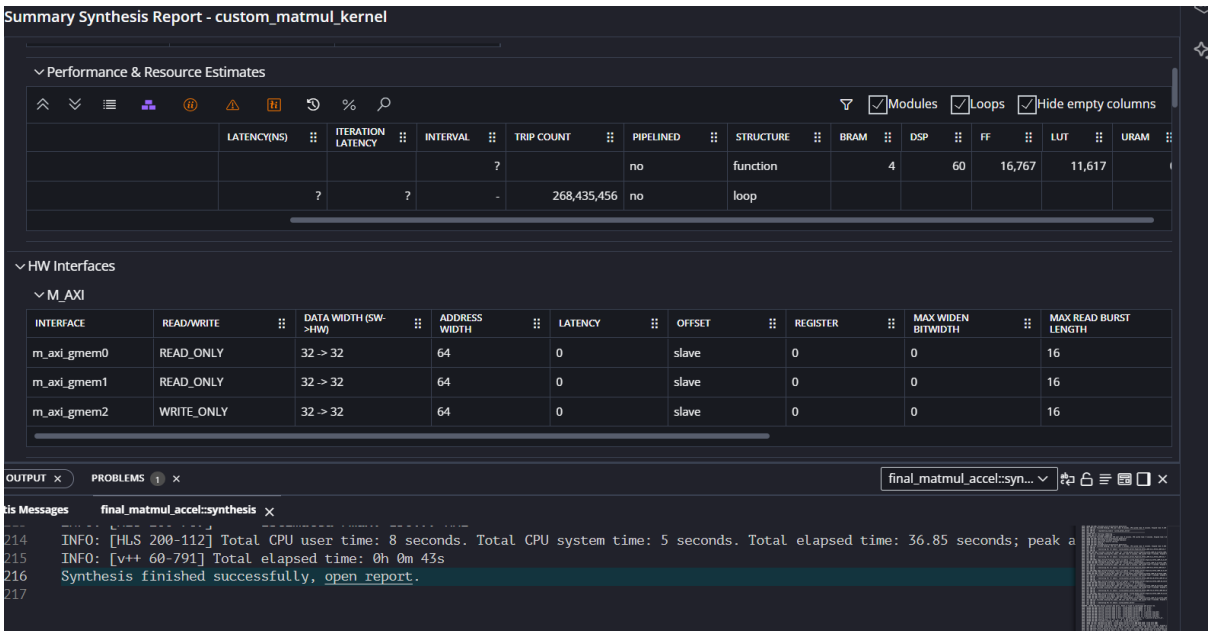

Figure 1: Vitis HLS synthesis report for the custom matrix multiplication accelerator.

Source Code Repository

https://github.com/braylonwatson/custom_accelerator/tree/main